

Apuntes de clase:
Computación Paralela y Distribuida
Unidad 3.4:
Fundamentos de Tipos de Datos MPI

Profesor: José Fiestas PhD
TA: Marcelo Zuloeta
Universidad de Ingeniería y Tecnología (UTEC)

12 de septiembre de 2026

1. Tipos de datos en MPI

En MPI, los mensajes no se describen únicamente por la cantidad de bytes que contienen. Cada operación de comunicación especifica:

(buffer, count, datatype)

donde:

- **buffer**: dirección inicial de memoria donde se encuentran los datos.
- **count**: número de elementos que serán enviados o recibidos.
- **datatype**: tipo MPI asociado a cada elemento.

Por ejemplo:

```
1 int A[100];  
2 MPI_Send(A, 100, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

En este caso se envían 100 elementos de tipo MPI_INT desde el proceso actual hacia el proceso 1

1.1. ¿Por qué MPI utiliza sus propios tipos de datos?

MPI debe funcionar en sistemas potencialmente heterogéneos, donde los procesos pueden ejecutarse sobre arquitecturas diferentes.

Por esta razón, MPI no describe los mensajes simplemente mediante cantidades de bytes, sino mediante tipos de datos. En el ejemplo anterior:

```
MPI_Send(A, 100, MPI_INT, ...);
```

indica que el mensaje contiene 100 enteros.

El sistema MPI puede utilizar esta información para interpretar correctamente los datos durante la comunicación.

Conceptualmente:

$$\text{Mensaje MPI} = \text{cantidad} \times \text{tipo de dato}$$

1.2. Tipos de datos básicos

MPI proporciona tipos predefinidos que corresponden a los tipos elementales de los lenguajes C y C++.

Tipo C/C++	Tipo MPI
char	MPI_CHAR
signed char	MPI_SIGNED_CHAR
unsigned char	MPI_UNSIGNED_CHAR
short	MPI_SHORT
unsigned short	MPI_UNSIGNED_SHORT
int	MPI_INT
unsigned int	MPI_UNSIGNED
long	MPI_LONG
unsigned long	MPI_UNSIGNED_LONG
float	MPI_FLOAT
double	MPI_DOUBLE
long double	MPI_LONG_DOUBLE

Cuadro 1: Correspondencia entre algunos tipos de C/C++ y MPI.

1.3. Ejemplo con un dato escalar

Supongamos que el proceso 0 desea enviar un entero al proceso 1.

```
1 int x = 20;
2
3 if (rank == 0)
4     MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
5
6 if (rank == 1)
7     MPI_Recv(&x, 1, MPI_INT, 0, 0,
8             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

El parámetro count=1 indica que solamente se transmite un elemento, o $1 \times MPI_INT$

1.4. Ejemplo con un vector

Considere el envío de un vector:

```
1 double A[100];
2 MPI_Send(A, 100, MPI_DOUBLE, destino, tag, MPI_COMM_WORLD);
```

El mensaje contiene 100 elementos de tipo MPI_DOUBLE y no es necesario realizar 100 operaciones MPI_Send.

Por tanto, una característica importante de MPI es que **una sola operación puede transmitir múltiples elementos del mismo tipo**.

1.5. Count y datatype

Los parámetros `count` y `datatype` deben interpretarse conjuntamente. En:

```
1 MPI_Send(A, 10, MPI_DOUBLE, . . .);
```

se envían 10 `MPI_DOUBLE`.

Mientras que en:

```
1 MPI_Send(A, 20, MPI_INT, ...);
```

se envían 20 `MPI_INT`.

Por lo tanto, `count` no indica el número de bytes transmitidos, sino el número de elementos del tipo especificado.

1.6. Tamaño de un tipo MPI

MPI permite consultar el tamaño asociado a un tipo mediante:

```
1 int size;
2 MPI_Type_size(MPI_DOUBLE, &size);
3 printf("MPI_DOUBLE ocupa %d bytes\n", size);
```

Si `size(MPI_DOUBLE)=8` entonces un mensaje:

```
1 MPI_Send(A, 100, MPI_DOUBLE, ...);
```

transporta aproximadamente $100 \times 8 = 800$ bytes de datos útiles.

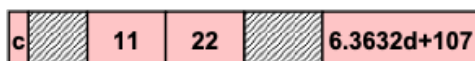
2. Tipos de datos derivados

Para garantizar la portabilidad del código en arquitecturas heterogéneas (donde la representación de datos o el tamaño en bytes de un `int` o `double` pueden variar), MPI no transmite datos como un flujo crudo de bytes, sino que requiere que el programador especifique el tipo de dato.

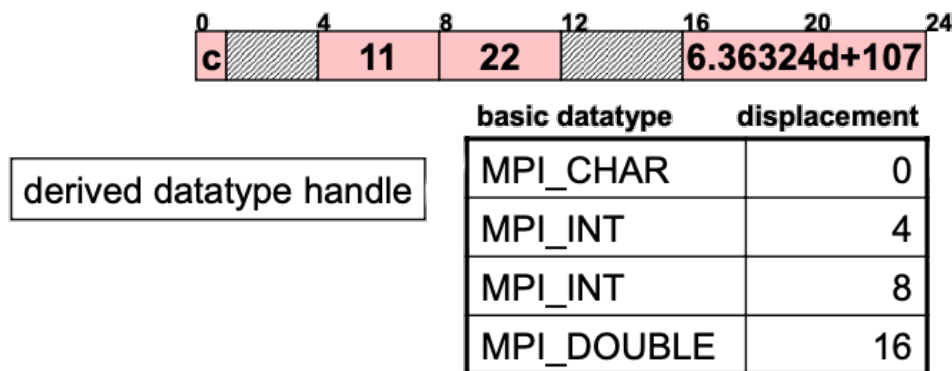
Son tipos construidos en base a los tipos básicos MPI (e.g. `MPI_INT`, `MPI_FLOAT`, ...)

Muchas veces se necesita:

- enviar data no-continua de un solo tipo (e.g. una submatriz) o distintos tipos
- enviar data continua de distintos tipos (e.g., un contador entero, seguido de doubles)



Tipos derivados MPI son punteros a una lista de entradas confirmadas por tipos básicos y desplazamientos. Estos describen la estructura de la data en memoria, e.g. de structs, subarrays



Pasos de implementacion de un Tipo Derivado:

1. **Construcción:** Llamar a una función constructora (ej. `MPI_Type_vector`).
2. **Confirmación (Commit):** Llamar a `MPI_Type_commit` antes de usarlo en comunicaciones.
3. **Uso:** Emplearlo en funciones como `MPI_Send` o `MPI_Bcast`.
4. **Liberación:** Llamar a `MPI_Type_free` para liberar recursos de memoria una vez que ya no se necesite.

MPI brinda herramientas para:

- no realizar múltiples llamadas MPI para enviar cada elemento (múltiples latencias)
- No copiar data en arrays contiguos (evitar comunicación)

2.1. El Problema de la Latencia de Red

Los programas reales no siempre trabajan con datos contiguos y homogéneos. Enviar un arreglo de 1000 elementos elemento por elemento (1000 llamadas a `MPI_Send`) es extremadamente ineficiente. El tiempo total de comunicación T_{comm} está dominado por la **latencia de inicio** (α) más que por el tiempo de transmisión por palabra (β)

$$T_{comm} = \alpha + x \cdot \beta \quad (1)$$

Reducir el número de mensajes es un objetivo relevante en el diseño paralelo. MPI ofrece herramientas para agrupar variables no contiguas o de distintos tipos en un solo mensaje.

Un tipo derivado en MPI describe un patrón o plantilla de memoria, compuesto por una secuencia de tipos básicos y sus desplazamientos (*displacements*) relativos en memoria. Pueden existir:

- estructuras,
- columnas de matrices,
- elementos separados regularmente,
- subconjuntos irregulares de un arreglo.

Un tipo derivado describe una determinada **distribución de datos en memoria**

2.2. Ventaja de los tipos derivados

Considere una matriz:

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix}.$$

En C, la matriz se almacena normalmente por filas:

$$a_{00}, a_{01}, a_{02}, a_{03}, a_{10}, a_{11}, \dots$$

Por tanto, una fila está almacenada de manera contigua.

Sin embargo, una columna como:

$$a_{01}, a_{11}, a_{21}, a_{31}$$

no se encuentra almacenada de forma contigua.

Un tipo derivado MPI permite describir directamente esta distribución.

3. MPI_Type_contiguous

Permite crear un nuevo tipo compuesto por varios elementos consecutivos del mismo tipo.

Su sintaxis general es:

int MPI_Type_contiguous (int count, MPI_Datatype old_type, MPI_Datatype * new_type);

Por ejemplo:

```
1 MPI_Datatype bloque;  
2 MPI_Type_contiguous(4, MPI_DOUBLE, &bloque);  
3 MPI_Type_commit(&bloque);
```

El nuevo tipo representa 4 MPI_DOUBLE consecutivos

Note que:

```
1 MPI_Send(A, 1, bloque, ...);
```

es equivalente a:

```
1 MPI_Send(A, 4, MPI_DOUBLE, ...);
```

3.1. MPI_Type_contiguous: ejemplo

$$P = \begin{bmatrix} x_0 & y_0 & z_0 \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{bmatrix} = \begin{bmatrix} P_0 \\ P_1 \\ P_2 \end{bmatrix}, \quad P_i = (x_i, y_i, z_i)$$

$$\underbrace{x_i \quad y_i \quad z_i}_{\text{punto3D}}$$

$$\text{punto3D} = \text{MPI_Type_contiguous}(3, \text{MPI_DOUBLE})$$

```

1 MPI_Datatype punto3D;
2
3 MPI_Type_contiguous(3, MPI_DOUBLE, &punto3D);
4 MPI_Type_commit(&punto3D);
5
6 MPI_Send(&P[i][0], 1, punto3D,
7         destino, tag, MPI_COMM_WORLD);
8
9 MPI_Type_free(&punto3D);

```

4. MPI_Type_vector

Permite representar datos separados regularmente en memoria.

La sintaxis es:

MPI_Type_vector (int count, int blocklength, int stride, MPI_Datatype datatype, MPI_Datatype *newtype);
donde:

- **count**: número de bloques.
- **blocklength**: número de elementos por bloque.
- **stride**: distancia entre el inicio de bloques consecutivos, en cantidad de elementos del tipo original.

4.1. Ejemplo: columna de una matriz

Considere:

```
double A[4][4];
```

Para describir una columna:

```

1 MPI_Datatype columna;
2 MPI_Type_vector(4, 1, 4, MPI_DOUBLE, &columna);
3 MPI_Type_commit(&columna);

```

Los parámetros son:

$$count = 4$$

porque existen cuatro elementos,

$$blocklength = 1$$

porque cada bloque contiene un elemento, y

$$stride = 4$$

porque existen cuatro elementos por fila.

La distribución seleccionada es:

$$A[0][j], A[1][j], A[2][j], A[3][j].$$

Ahora la columna puede transmitirse directamente:

```

1 MPI_Send(&A[0][j], 1, columna, destino, tag, MPI_COMM_WORLD);

```

5. Tipos con distribución irregular de datos en memoria

5.1. MPI_Type_indexed

Cuando los datos no presentan una separación regular, puede utilizarse:

MPI_Type_indexed (int count, int blocklengths[], int displacements[], MPI_Datatype datatype, MPI_Datatype *newtype);

count: cantidad de bloques

blocklengths: array conteniendo la longitud de cada bloque

displacements: array conteniendo las distancias de cada bloque desde el inicio al inicio de cada bloque

datatype: el tipo básico MPI

newtype: variable que almacena el tipo generado

Este mecanismo permite especificar:

- diferentes tamaños de bloques,
- diferentes desplazamientos.

Conceptualmente se pueden seleccionar posiciones como:

$$A[0], A[3], A[4], A[10], A[15]$$

sin que exista necesariamente una separación constante entre ellas.

Este tipo es útil para representar patrones irregulares de acceso a memoria.

6. MPI_Type_create_struct

Una de las aplicaciones más importantes de los tipos derivados consiste en transmitir estructuras de C.

Considere:

```
struct Particula {  
    int id;  
    double x;  
    double y;  
    double z;  
};
```

Una alternativa poco conveniente sería realizar cuatro comunicaciones:

```
1 MPI_Send(&p.id, 1, MPI_INT, ...);  
2 MPI_Send(&p.x, 1, MPI_DOUBLE, ...);  
3 MPI_Send(&p.y, 1, MPI_DOUBLE, ...);  
4 MPI_Send(&p.z, 1, MPI_DOUBLE, ...);
```

MPI permite construir un único tipo que represente toda la estructura.

6.1. Construcción del tipo

Se definen los tipos de cada campo, la longitud de cada bloque, y los desplazamientos de los campos dentro de la estructura.

El envío de estructuras en C (heterogéneas) introduce un problema crítico: el **Padding** o Alineación de Memoria. Los compiladores insertan bytes vacíos entre las variables de un **struct** para alinear el acceso a memoria del procesador. Por esto, los desplazamientos no pueden adivinarse matemáticamente. Para crear un tipo derivado de una estructura segura, siempre se debe consultar la dirección física de las variables en tiempo de ejecución

```
1 MPI_Datatype tipos[4] = {MPI_INT, MPI_DOUBLE, MPI_DOUBLE, MPI_DOUBLE};
2 int bloques[4] = {1,1,1,1};
3 MPI_Aint desplazamientos[4];
4 MPI_Get_address(&p.id, &desplazamientos[0]);
5 MPI_Get_address(&p.x, &desplazamientos[1]);
6 MPI_Get_address(&p.y, &desplazamientos[2]);
7 MPI_Get_address(&p.z, &desplazamientos[3]);
```

Los desplazamientos deben expresarse respecto a una dirección base:

```
1 MPI_Aint base;
2 MPI_Get_address(&p, &base);
3
4 for (int i = 0; i < 4; i++)
5     desplazamientos[i] -= base;
6
7 MPI_Datatype MPI_PARTICULA;
8
9 MPI_Type_create_struct(4, bloques, desplazamientos, tipos, &
10     MPI_PARTICULA);
11 MPI_Type_commit(&MPI_PARTICULA);
```

Listing 1: Ejemplo de construcción de tipo Struct

Y se envía la estructura mediante:

```
1 MPI_Send(&p, 1, MPI_PARTICULA, destino, tag, MPI_COMM_WORLD);
```

7. Commit de un tipo

Después de crear un tipo derivado debe registrarse antes de utilizarlo:

MPI_Type_commit (&tipo);

Por ejemplo:

```
1 MPI_Datatype columna;
2 MPI_Type_vector(N, 1, N, MPI_DOUBLE, &columna);
3 MPI_Type_commit(&columna);
```

Solo después del **commit** debe utilizarse en las operaciones de comunicación.

8. Liberación de un tipo

Cuando el tipo derivado deja de ser necesario debe liberarse:

MPI_Type_free (&tipo);

Por ejemplo:

```
1 MPI_Type_free(&columna);
```

9. Tipos derivados y empaquetamiento

Sin tipos derivados, cuando los datos no son contiguos *podría ser necesario copiarlos previamente a un buffer*. Utilizando tipos derivados **MPI recibe la descripción de la distribución de memoria y se encarga de acceder a los elementos correspondientes**

Los tipos derivados permiten separar **qué datos se comunican** de **cómo están almacenados**

10. Ejemplo aplicado: intercambio de bordes

Los tipos derivados son especialmente importantes en algoritmos paralelos sobre matrices.

Supongamos una matriz distribuida entre procesos:

$$\begin{aligned} P_0 &: \text{filas } 0 \dots k - 1 \\ P_1 &: \text{filas } k \dots 2k - 1 \\ P_2 &: \text{filas } 2k \dots 3k - 1 \end{aligned}$$

En un suavizado 2D o método de diferencias finitas puede ser necesario intercambiar bordes entre procesos vecinos.

Si el dominio se divide por filas, una fila completa es contigua y puede enviarse directamente:

```
1 MPI_Send(&A[filas][0], N, MPI_DOUBLE, ...);
```

Si el dominio se divide por columnas, una columna no es contigua.

En ese caso puede definirse:

```
1 MPI_Type_vector(N, 1, N, MPI_DOUBLE, &columna);
2 MPI_Type_commit(&columna);
3
4 MPI_Send(&A[0][j], 1, columna, ...);
```

Este mecanismo evita copiar manualmente la columna a un buffer auxiliar.

11. Empaquetado de Datos: MPI_Pack y MPI_Unpack

Una alternativa a construir tipos estructurados formales es empaquetar variables secuencialmente en un arreglo de bytes (**MPI_PACKED**) antes de enviarlo.

- **Ventaja:** Es extremadamente dinámico. Ideal cuando la estructura del mensaje varía en tiempo de ejecución o es una comunicación de un solo uso.

- **Desventaja:** Incurrir en un costo extra por copiar explícitamente los datos de las variables al *buffer* de empaquetado por parte de la CPU (overhead de memoria).

El `MPI_Type_create_struct` es preferible cuando la comunicación del mismo tipo de estructura se realiza de forma recurrente dentro de un bucle.

Empaqueta distintos tipos de datos en memoria

int MPI_Pack (void *inbuf, int incout, MPI_Datatype datatype, void *outbuf, int outsize, int *position, MPI_Comm comm);

inbuf: Inicio del buffer de entrada.

incout: Numero de datos de entrada.

datatype: Tipo de dato de los datos de entrada.

outsize: Tamaño del buffer de salida, en bytes.

comm: Comunicador para el mensaje empaquetado.

position: Actual posición del buffer, en bytes.

int MPI_Unpack (const void* inbuf, int insize, void *outbuf, int outcount, int& position, const Comm& comm) const;

```

1  #define TAM_BUFFER 100
2  #define TAG 0
3  #define DEST 1
4  #define SOURCE 0
5  int main() {
6  char  buffer[TAM_BUFFER];
7  int    my_rank, posicion,n;
8  float a, b;
9  MPI_Status  status;
10 MPI_Init(&argc, &argv);
11 MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
12 if ( rank == SOURCE ) {
13     a = 10.3;
14     b = 11.2
15     n = 4;
16     posicion = 0;
17     MPI_Pack(&a,1,MPI_FLOAT,buffer,TAM_BUFFER,&posicion,MPI_COMM_WORLD
18 );
19     MPI_Pack(&b,1,MPI_FLOAT,buffer,TAM_BUFFER,&posicion,MPI_COMM_WORLD
20 );
21     MPI_Pack(&n,1,MPI_INT,buffer,TAM_BUFFER,&posicion,MPI_COMM_WORLD);
22     MPI_Send(buffer,TAM_BUFFER,MPI_PACKED,DEST,TAG,MPI_COMM_WORLD);
23 }
24 else {
25     MPI_Recv(buffer,TAM_BUFFER,MPI_PACKED,SOURCE,TAG,
26 MPI_COMM_WORLD, &status);
27     posicion = 0;
28     MPI_Unpack(buffer,TAM_BUFFER,&posicion,&a,1,MPI_FLOAT,
29 MPI_COMM_WORLD);
30     printf("    a = %f\n",a);
31     MPI_Unpack(buffer,TAM_BUFFER,&posicion,&b,1,MPI_FLOAT,
32 MPI_COMM_WORLD);
33     printf("    b = %f\n",b);
34     MPI_Unpack(buffer,TAM_BUFFER,&posicion,&n,1,MPI_INT,
35 MPI_COMM_WORLD);

```

```

34     printf("    n = %d\n",n);
35 }
36 MPI_Finalize();
37 }

```

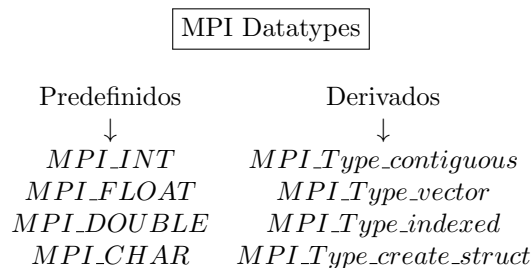
12. Operaciones Avanzadas y Utilidades

12.1. MPI_Sendrecv

Es una función diseñada específicamente para enviar y recibir un mensaje simultáneamente, operando internamente de manera segura. Es la solución óptima y nativa para prevenir **deadlocks** en topologías cíclicas (como intercambio de fronteras en anillos 1D), reemplazando la necesidad de ordenar lógicamente pares de `MPI_Send` y `MPI_Recv`.

13. Resumen

Los tipos MPI pueden dividirse en dos grandes grupos:



Los tipos predefinidos permiten transmitir datos elementales y arreglos contiguos.

Los tipos derivados permiten describir estructuras de memoria más complejas, como:

- bloques contiguos,
- columnas de matrices,
- patrones irregulares,
- estructuras heterogéneas.

Los tipos MPI (MPI Datatype) son una **descripción lógica de los datos que forman un mensaje**

Referencias

- [1] <https://www.mpich.org/documentation/guides/>
- [2] William Gropp, Ewing Lusk and Anthony Skjellum: *Using MPI: Portable Parallel Programming With the Message-Passing Interface*. MIT Press, 3rd edition, Nov. 2014 (336 pages, ISBN 9780262527392)
- [3] William Gropp, Torsten Hoefler, Rajeev Thakur and Ewing Lusk: *Using Advanced MPI: Modern Features of the Message-Passing Interface*. MIT Press, Nov. 2014 (392 pages, ISBN 9780262527637)
- [4] Neil MacDonald, Elspeth Minty, Joel Malard, Tim Harding, Simon Brown, Mario Antonioletti: *Parallel Programming with MPI. Historical MPI course notes from EPCC*. http://www.archer.ac.uk/training/course-material/2014/10/MPI_UCL/Notes/MPP-notes.pdf
- [5] Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5. Ch 3
- [6] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656. Ch 4